

Storing Data

Many of your programs will ask users to input certain kinds of information. You might allow users to store preferences in a game or provide data for a visualization. Whatever the focus of your program is, you'll store the information users provide in data structures such as lists and dictionaries. When users close a program, you'll almost always want to save the information they entered. A simple way to do this involves storing your data using the `json` module.

The `json` module allows you to convert simple Python data structures into JSON-formatted strings, and then load the data from that file the next time the program runs. You can also use `json` to share data between different Python programs. Even better, the JSON data format is not specific to Python, so you can share data you store in the JSON format with people who work in many other programming languages. It's a useful and portable format, and it's easy to learn.

NOTE

The JSON (JavaScript Object Notation) format was originally developed for JavaScript. However, it has since become a common format used by many languages, including Python.

Using `json.dumps()` and `json.loads()`

Let's write a short program that stores a set of numbers and another program that reads these numbers back into memory. The first program will use `json.dumps()` to store the set of numbers, and the second program will use `json.loads()`.

The `json.dumps()` function takes one argument: a piece of data that should be converted to the JSON format. The function returns a string, which we can then write to a data file:

```
number
_writer.py    from pathlib import Path
               import json

               numbers = [2, 3, 5, 7, 11, 13]

❶ path = Path('numbers.json')
❷ contents = json.dumps(numbers)
  path.write_text(contents)
```

We first import the `json` module, and then create a list of numbers to work with. Then we choose a filename in which to store the list of numbers ❶. It's customary to use the file extension `.json` to indicate that the data in the file is stored in the JSON format. Next, we use the `json.dumps()` ❷ function to generate a string containing the JSON representation of the data we're working with. Once we have this string, we write it to the file using the same `write_text()` method we used earlier.

This program has no output, but let's open the file `numbers.json` and look at it. The data is stored in a format that looks just like Python:

```
[2, 3, 5, 7, 11, 13]
```
